

Algoritmos distribuidos para anillos

Algoritmos Distribuidos



UNIVERSIDAD AUTÓNOMA METROPOLITANA
UNIDAD IZTAPALAPA

Problema: Exclusión mutua

- Considere un sistema de n procesos
 - $P_1, P_2, \dots, P_n$
- Los segmentos de código en los cuales un proceso manipula un recurso compartido de uso exclusivo se denominan **secciones críticas**.
- Para no tener resultados erróneos, la sección crítica se debe ejecutar en *exclusión mutua*: es decir, sólo un proceso a la vez está ejecutando su sección crítica.

Conceptos básicos: Exclusión mutua

- En toda solución pesimista hay una fase de entrada en donde se solicita el permiso de ejecución y una fase de salida en donde se notifica al sistema la finalización de la sección crítica.

ADQUIERE DERECHO DE ENTRADA
EJECUTA SECCIÓN CRÍTICA
NOTIFICA TERMINACIÓN

Problema: Exclusión mutua

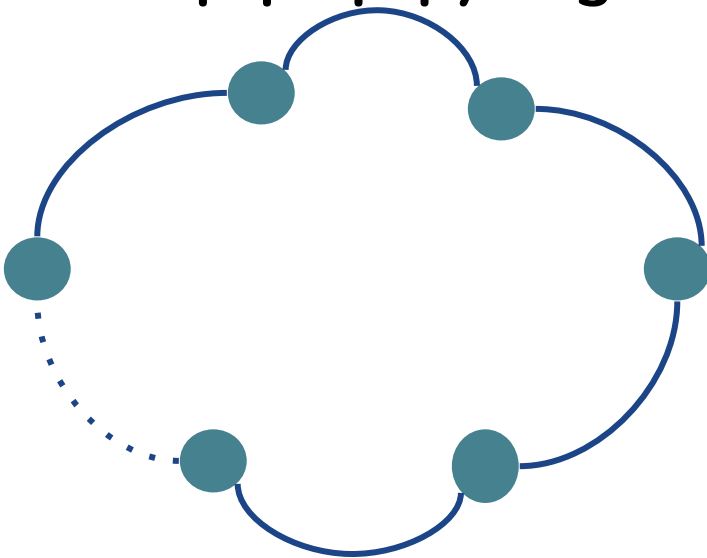
- **Suponiendo que:**
 - Un proceso ejecuta su sección crítica en un tiempo finito
 - Si un proceso se detiene fuera de la sección crítica no interfiere con los otros procesos
 - No se tienen restricciones ni conocimiento acerca de las velocidades relativas de los procesos.
- **Una solución al problema de la sección crítica debe cumplir los siguientes requisitos**
 - Exclusión mutua: Si P_i está ejecutando su sección crítica, ningún otro proceso está ejecutando su sección crítica
 - Espera limitada: Nadie debe esperar indefinidamente el entrar en la sección crítica.
 - Progreso: Cuando nadie está ejecutando la sección crítica y un proceso solicita la entrada, se le debe conceder sin retardo.

Algoritmo para exclusión mutua en un anillos

Modelos:

Sistema con topología de anillo:

- Grafo conexo que consiste de un ciclo único que contiene a todos los vértices.
- $|V| = |A|$ y el grado de cada vértice es 2.



Dos tipos:

- » Unidireccional: Los mensajes circulan en una sola dirección.
- » Bidireccional: Los mensajes circulan en ambas direcciones.

Modelos

- Tiempo: asíncrono
- Comunicación: por intercambio de mensajes
- Fallas: los nodos no fallan y los mensajes llegan siempre íntegros a su destino con un retardo finito no acotado.

Algoritmo para exclusión mutua en un anillos

Estrategia:

- El derecho de acceso está representado por un token.
- El token circula en el anillo en una sola dirección.
- Un nodo que desea ejecutar su sección crítica debe esperar a que le llegue el token, entonces lo retiene, ejecuta su sección crítica y cuando termina, circula el token.



Algoritmo para exclusión mutua en un anillos

Variables de estado:

- **sucesor**: nodo al que se le pueden enviar mensajes. Conocido desde el inicio.
- **solicitud_sc**: bandera cuyo valor es **VERDADERO** si existe una solicitud de la aplicación local para ejecutar la SC y **FALSO** en otro caso. Inicialmente es **FALSO**.

Mensajes del algoritmo distribuido de EM:

- **Token**:
 - Inicialmente lo genera un único nodo.
 - Circula siempre del nodo a su sucesor.



Algoritmo para exclusión mutua en un anillos

Existen además, mensajes que se intercambian entre la instancia local del AD de EM y la aplicación local.

- SOLICITUD: la aplicación local se lo envía al algoritmo distribuido local de EM cuando requiere ejecutar su sección crítica.
- OK: el algoritmo distribuido local de EM lo envía a la aplicación cuando el token está en el nodo.
- LIBERA: la aplicación local lo envía al algoritmo distribuido local de EM cuando termina de ejecutar su sección crítica.



Algoritmo para exclusión mutua en el nodo i

DLOCK()

/* Sección crítica */

DUNLOCK

DLOCK()

Envía SOLICITUD al AD local

Espera OK desde al AD local

UNLOCK()

Envía LIBERA al AD local

Algoritmo para exclusión mutua en el nodo i

Al recibir SOLICITUD desde la aplicación

solicitud_sc = Verdad

Al recibir TOKEN en el nodo i:

SI (NO solicitud_sc) ENTONCES

ENVIA token a sucesor

OTRO

Envía OK a la aplicación

FINSI

Al recibir LIBERA desde la aplicación

Solicitud_sc = FALSO

ENVIA token a sucesor

/* Se asume que solo puede existir una solicitud pendiente en un instante dado */

Propiedades

- **La exclusión mutua está garantizada:**
 - Solo existe un token $\Rightarrow$ solo un nodo puede poseerlo $\Rightarrow$ solo un nodo puede ejecutar su sección crítica.
- **Espera limitada se garantiza dado que:**
 - el tiempo de ejecución de la SC es finito,
 - la transmisión del token requiere tiempo finito
 - y el token se transmite un número finito de veces

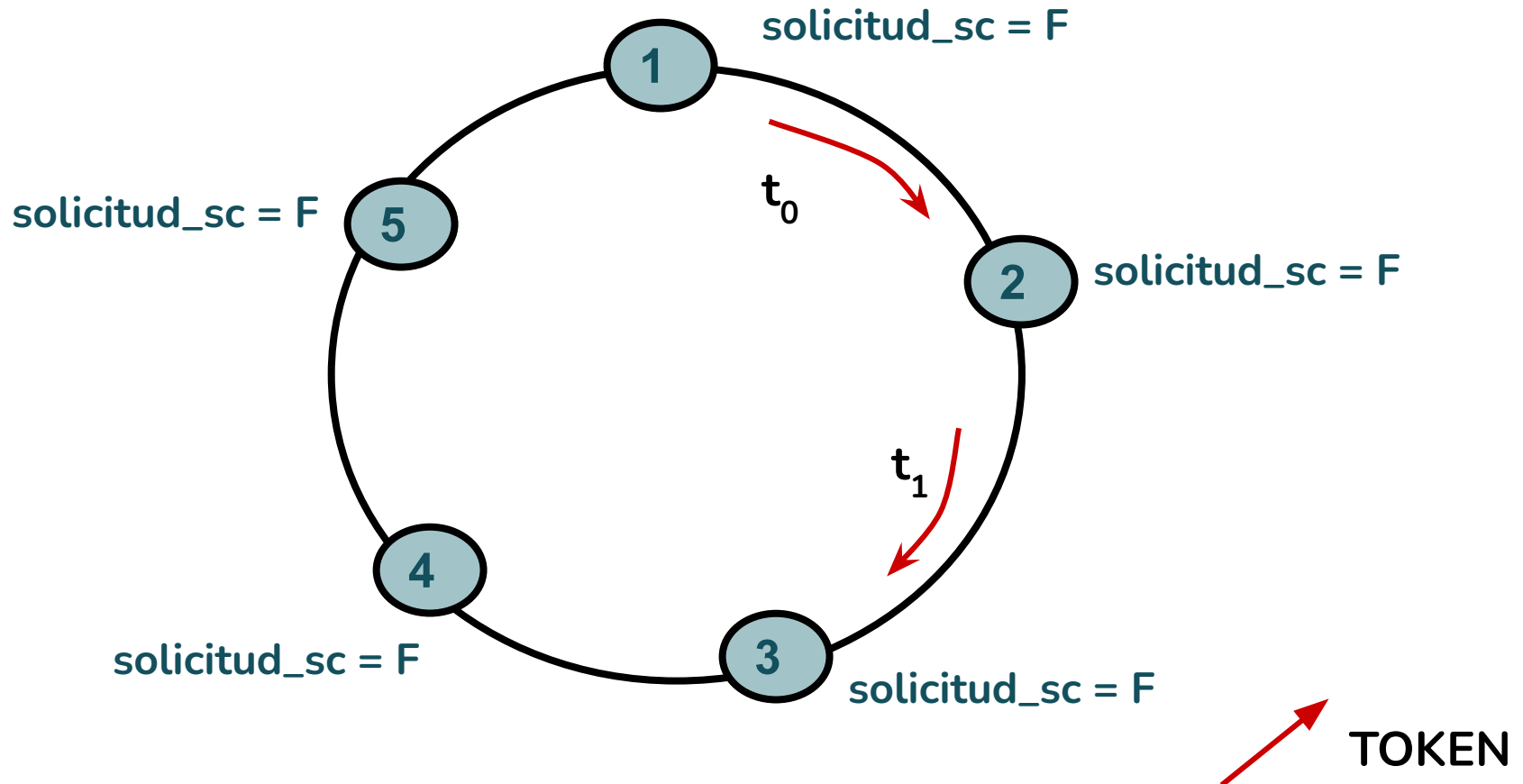
$\Rightarrow$ un nodo espera tiempo finito
- **Progreso se garantiza** pues si solo un nodo solicita el derecho de acceso, los demás reenvían el token y llega al nodo que lo requiere después de un número finito de mensajes.



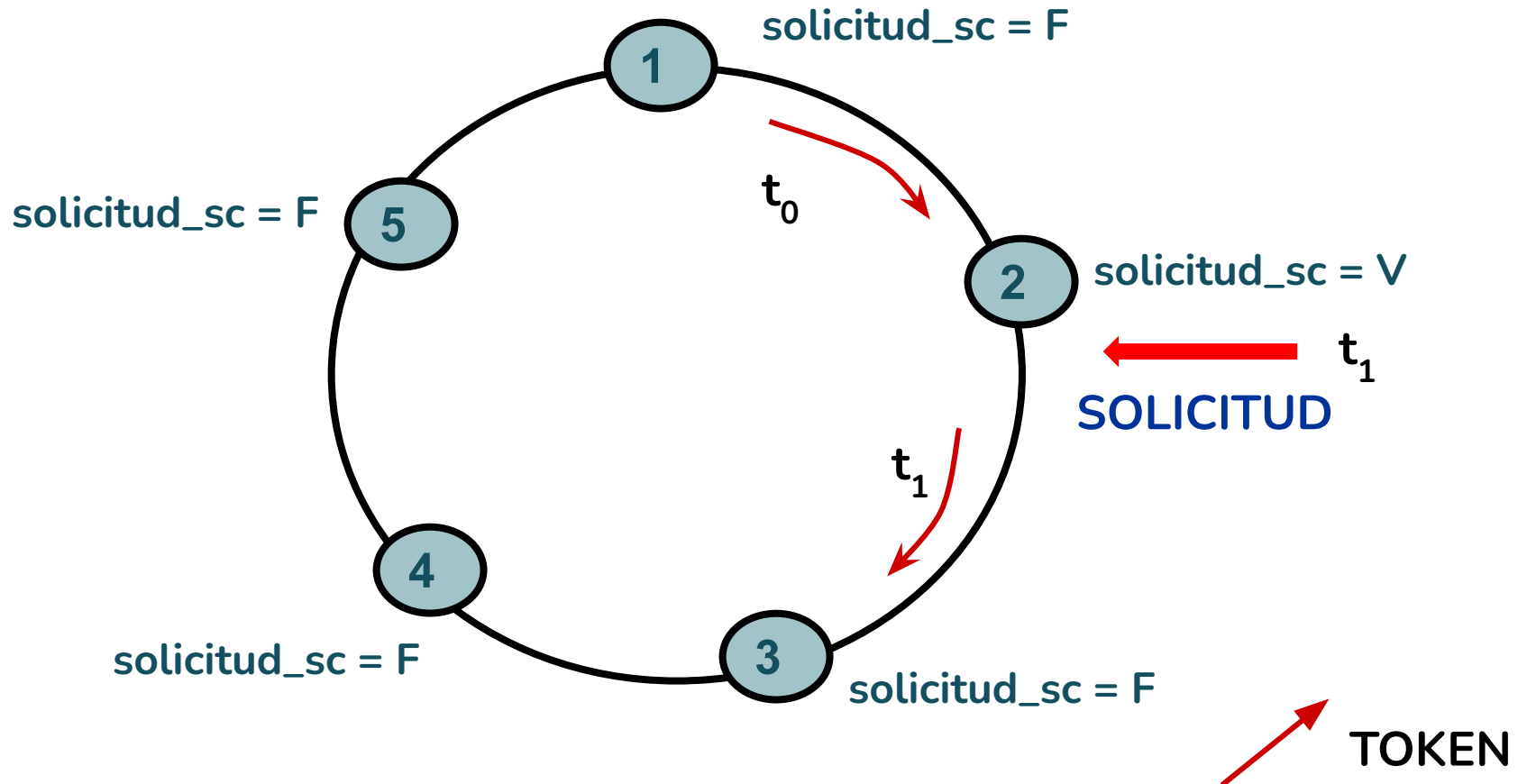
Desventajas

- Aun cuando nadie solicite ejecutar la SC, el token siempre circula
- La espera por el token puede durar entre 0 y $n-1$ transmisiones
- En un escenario con fallas es necesario:
 - Garantizar la existencia de un y sólo un token
 - Garantizar la integridad del anillo.

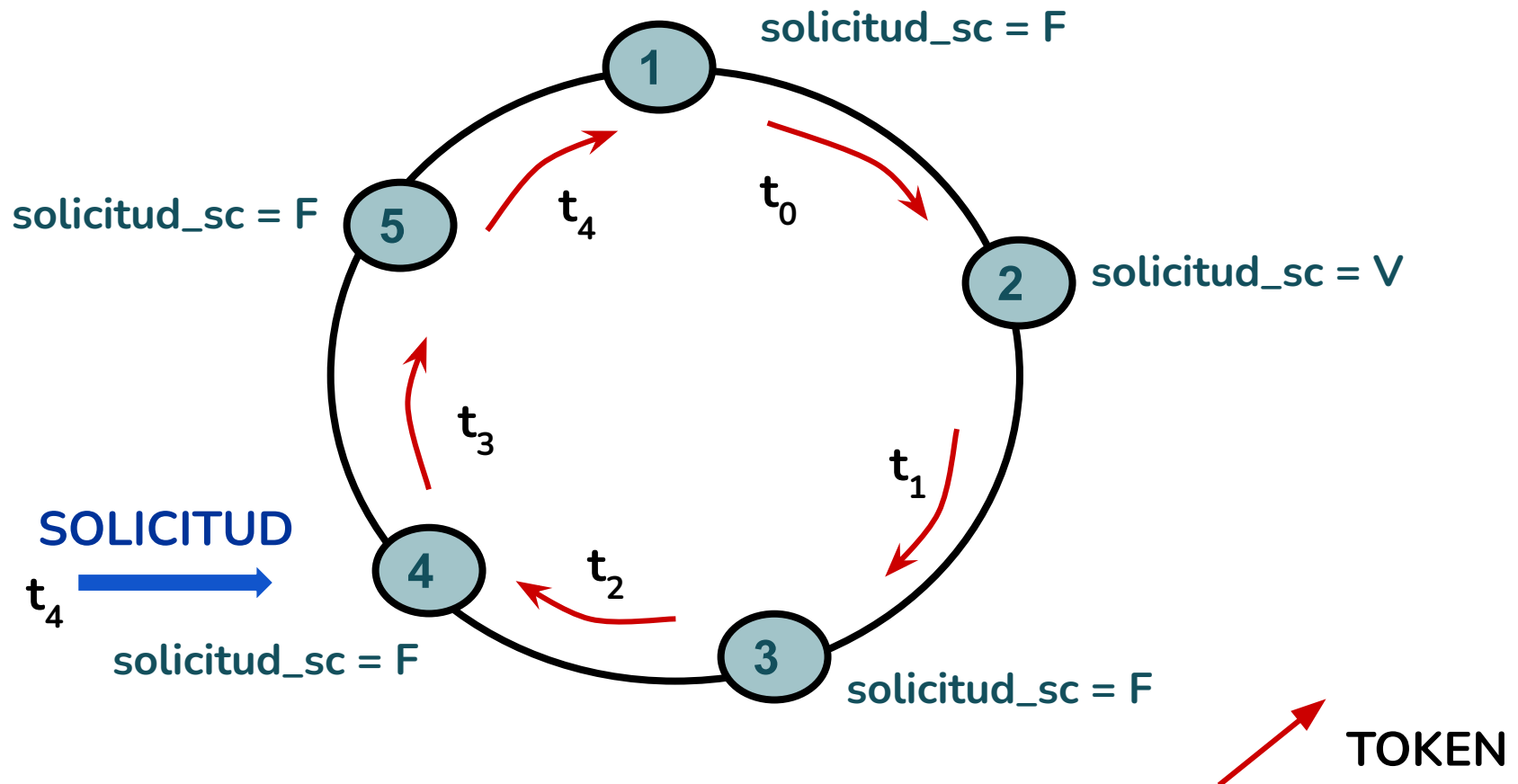
Ejemplo



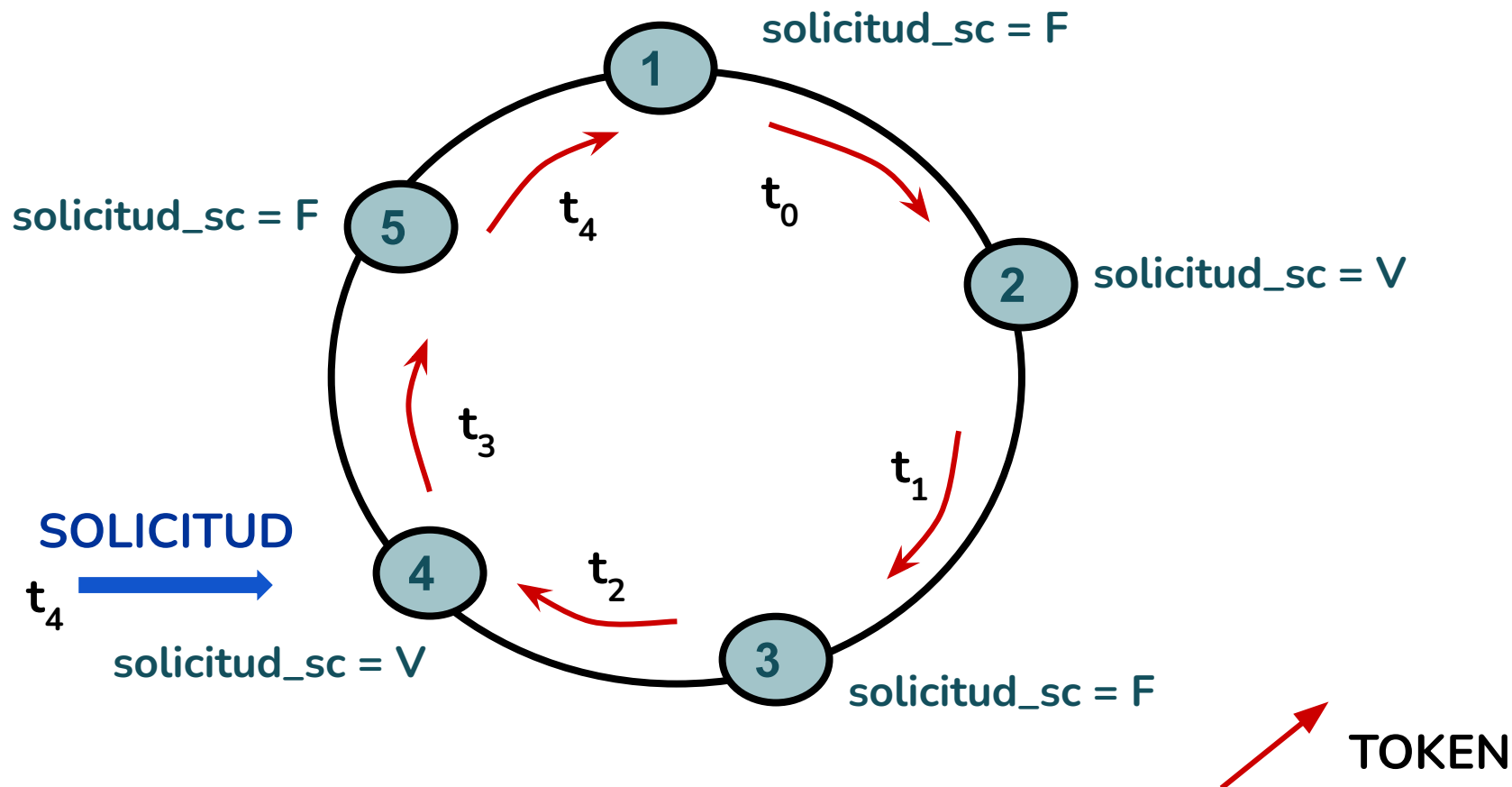
Ejemplo



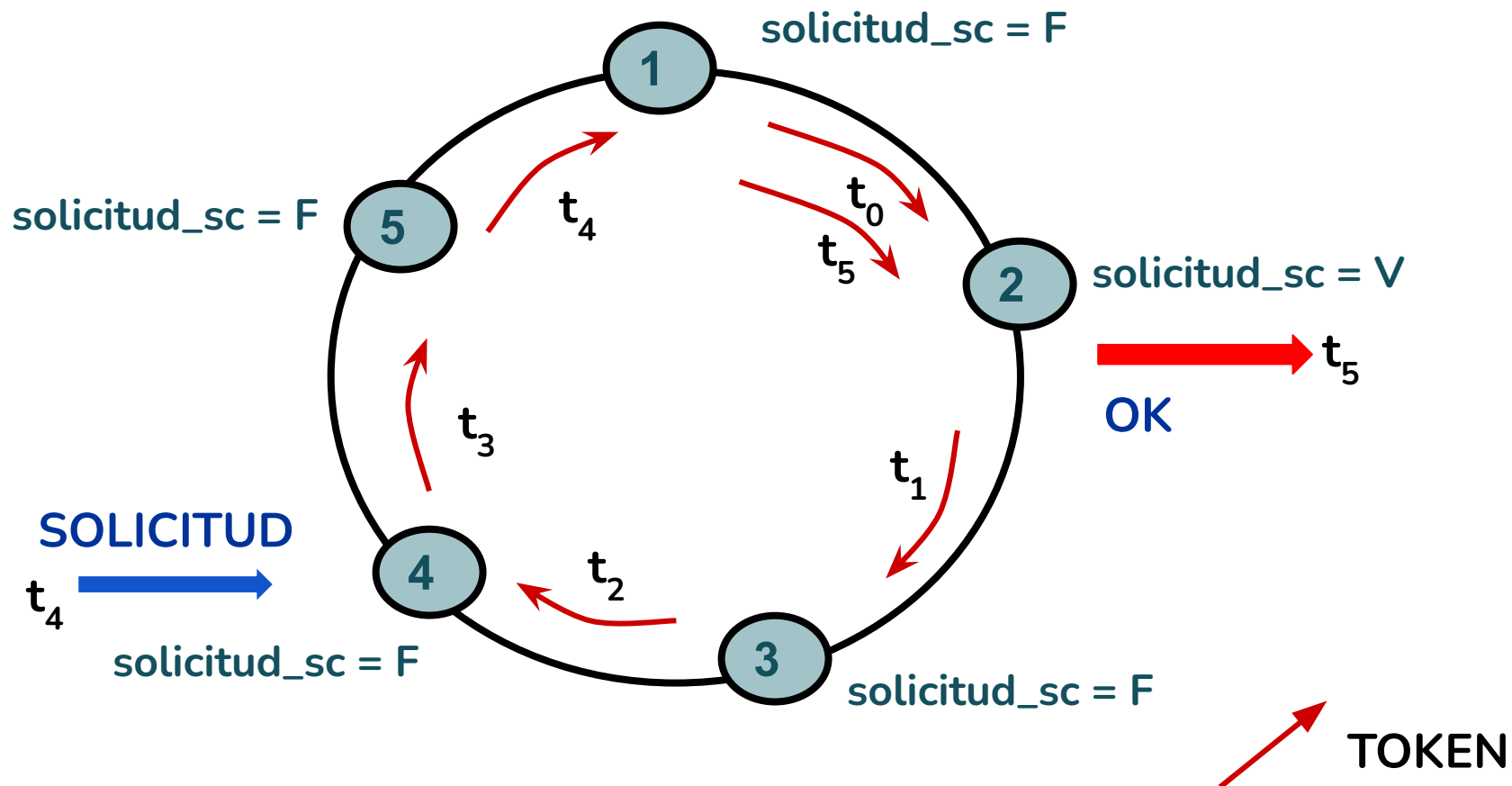
Ejemplo



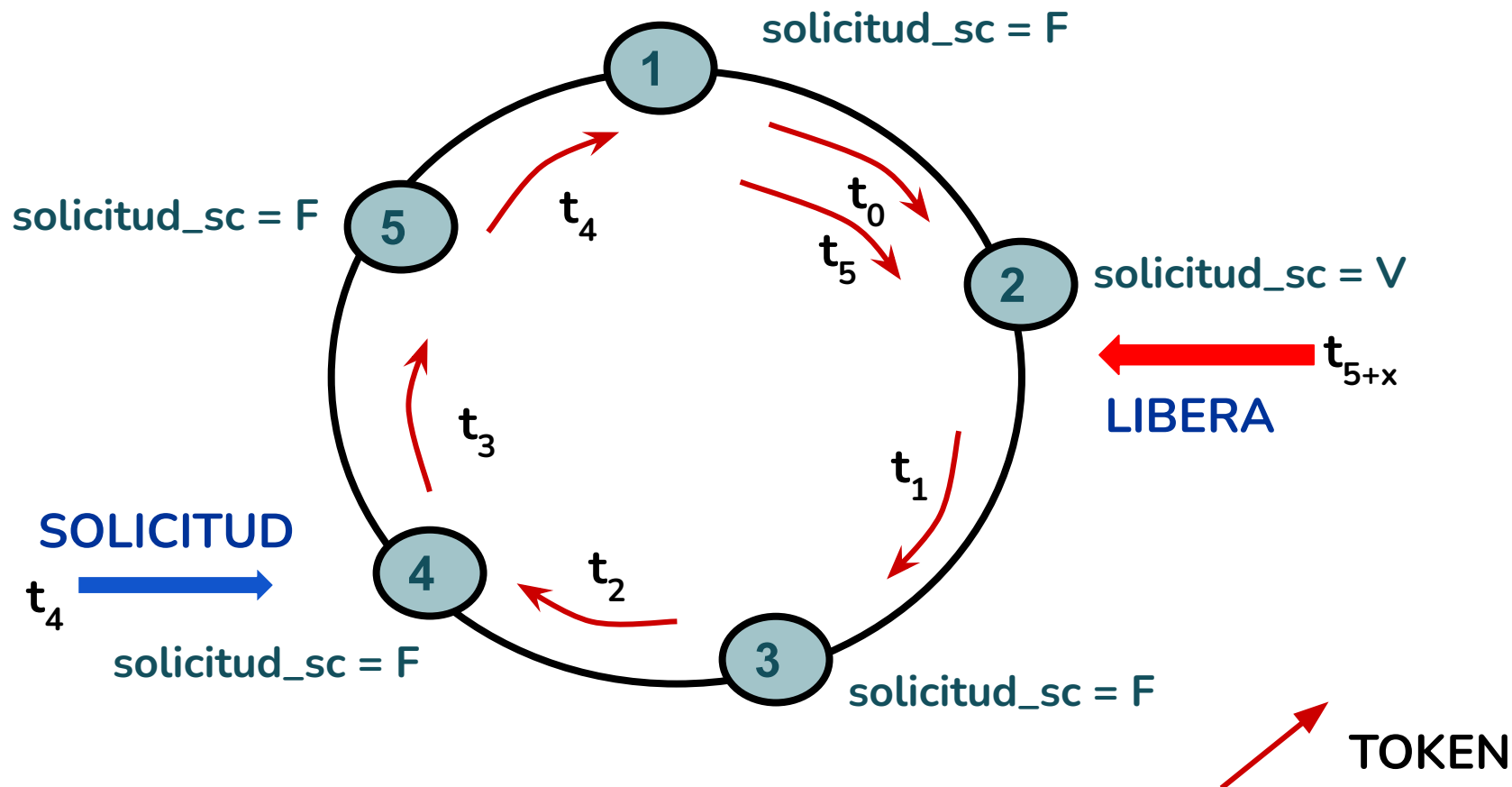
Ejemplo



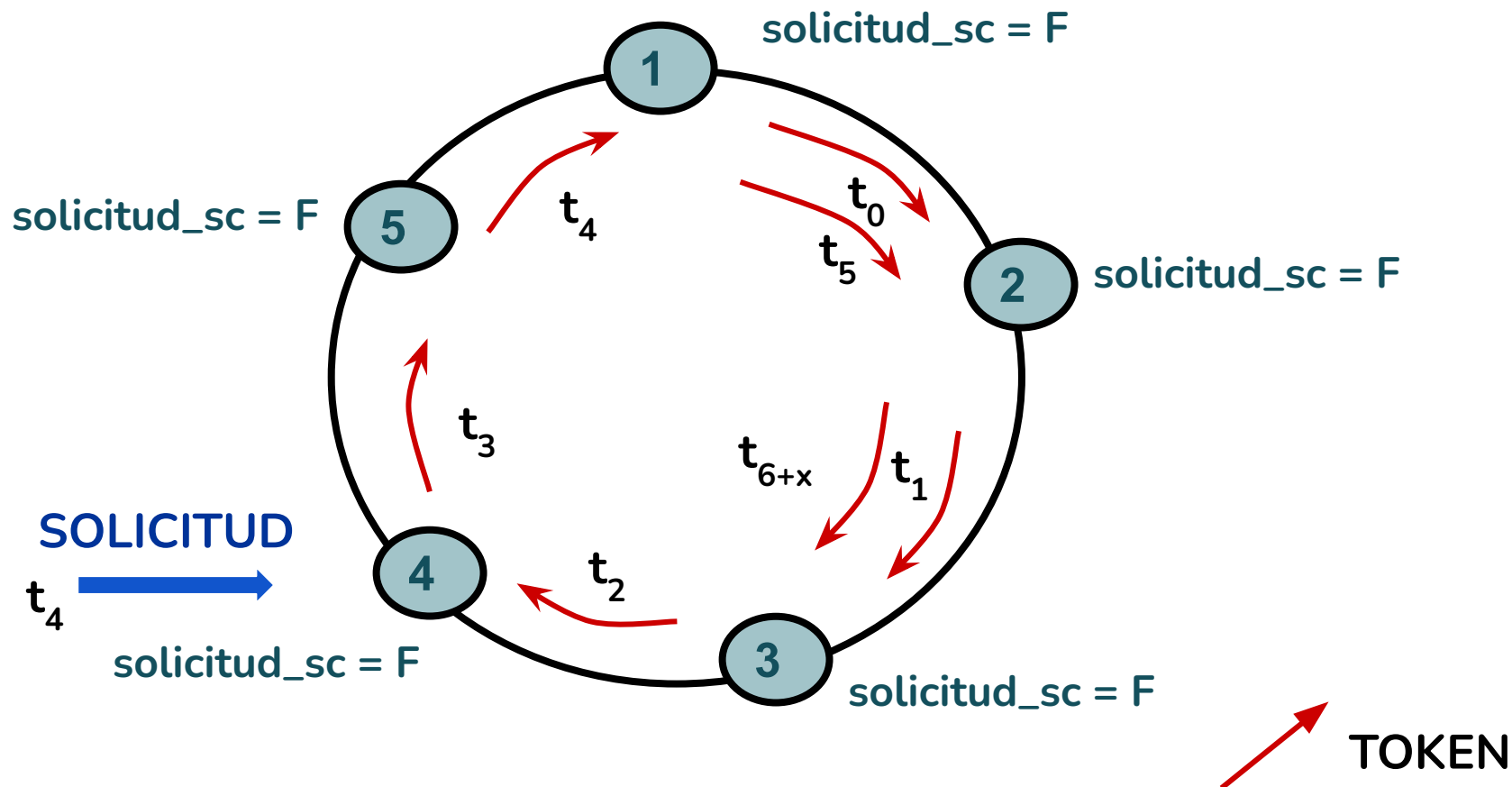
Ejemplo



Ejemplo



Ejemplo



Ejercicio

Programar ambos algoritmos en el simulador



Elección

- En muchos problemas del cómputo distribuido se requiere que un nodo juegue un rol especial. Llamaremos a ese nodo el **líder**.
- Si el líder por alguna razón deja de funcionar como tal, otro nodo, sólo uno, debe asumir el rol especial.
- Se necesita pues tener un algoritmo para **elegir** al nuevo líder.



Propiedades de un algoritmo de elección

- A lo más un nodo es electo líder (seguridad: no pasa nada malo)
- En un tiempo finito todos los procesos entran en un estado final y un nodo es electo líder (vivacidad: lo bueno sucede en algún momento)

Algoritmo de Le Lann, Chang y Roberts

Estrategia:

- En los algoritmos revisados se elige al candidato i con el mayor identificador (pero este criterio puede ser sustituido)
- Un nodo interesado en volverse líder lanza su candidatura a recorrer la red.
- Cada nodo que ve pasar la candidatura la propaga si el candidato es más fuerte que él mismo. Si no es el caso, elimina la candidatura recibida y lanza la suya.



Algoritmo de Le Lann, Chang y Roberts

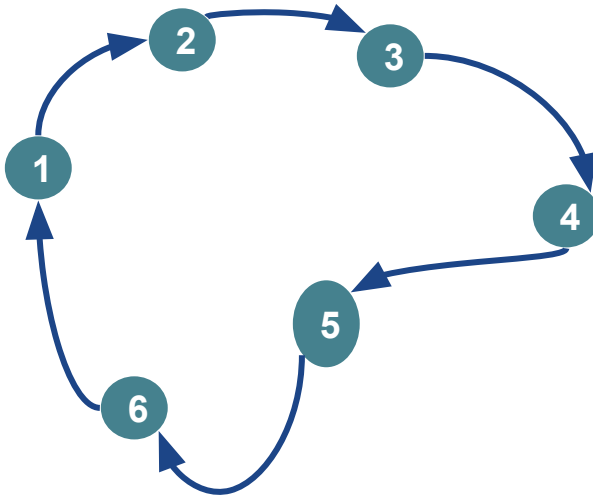
Estrategia:

- Si una candidatura llega al nodo que la lanzó después de haber pasado por todos los nodos, significa que en el camino no se encontró ningún candidato más fuerte y por lo tanto el nodo es el nuevo líder.
- El nuevo líder ahora debe anunciar a todos los nodos de la red informando que es el ganador de la contienda.



Modelo

- Grafo dirigido con topología de anillo $\Rightarrow$ los mensajes circulan en una sola dirección, en el sentido de las manecillas del reloj.



Sobre cualquier topología física se puede construir una topología lógica de anillo asignando a cada nodo un sucesor y un predecesor.

Estado de un proceso

Cada proceso conserva las siguientes variables de estado:

- **estado**: indica el estado en el que se encuentra el nodo. Puede tener los valores *despierto* o *dormido*. Inicialmente tiene el valor *dormido*.
- **líder**: al final de la elección almacena el identificador del nodo electo. Inicialmente contiene un valor no usado como id de ningún nodo. En nuestro caso usaremos **0**.



Mensajes

- **Candidatura(j)**: Mensaje que transporta la identidad de un candidato j . Se origina en el nodo j cuando lanza su candidatura.
- **Electo(k)**: Mensaje que transporta la identidad del ganador de la contienda. Se origina en el nodo k .

Algoritmo Elección LCR en el nodo i

Al recibir CANDIDATURA(j) desde la izquierda HAZ
SI ($j > i$) ENTONCES Envía CANDIDATURA(j) a la derecha
OTRO
 SI ($i > j$) Y (estado $\neq$ despierto) ENTONCES
 Envía CANDIDATURA(i) a la derecha
 OTRO SI ($j = i$) ENTONCES
 Envía ELECTO(i) a la derecha
FINSI
estado = despierto



Algoritmo Elección LCR en el nodo i

Al recibir `Electo(j)` desde la izquierda HAZ

`lider = j`

SI `(j != i)` ENTONCES

 Envía `Electo(j)` a la derecha

`estado = inactivo`



Ejemplo de ejecución

Propiedades

Lema 1. Todos los procesos del anillo terminan el algoritmo. Cuando esto ocurre, existe exactamente un proceso al que todos reconocen como líder.

Dem.

- 1) Los ids de los nodos son únicos y entre ellos existe una relación de orden total, entonces existe solo un nodo con identificador k tal que $j < k$ para todo j identificador de otro nodo.
- 2) Todos los nodos se activan en algún momento. Solos o porque reciben una candidatura.
- 3) En particular el nodo k se activa y envía su candidatura.



Propiedades

- 4) El mensaje con la candidatura de k es reenviado por todos los nodos y regresa a k . k se sabe líder y genera el mensaje $\text{Electo}(k)$.
- 5) Cada nodo $j \neq k$ recibe $\text{Electo}(k)$, registra a k como líder, reenvían el mensaje $\text{Electo}(k)$ y termina.
- 6) Cuando el mensaje $\text{Electo}(k)$ llega a k , ya no lo envía y se da por terminado el algoritmo.
- 7) Todos terminan y el nodo k es reconocido por todos como el líder.

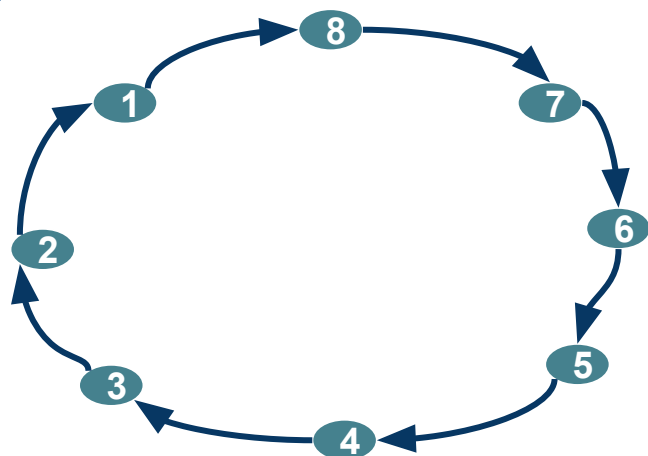


Complejidad de comunicaciones

Lema 2. En el peor caso, el costo en mensajes del algoritmo LCR tiene una complejidad $O(n^2)$

Dem. El peor caso ocurre cuando todos los nodos se despiertan autónomamente y lanzan su candidatura. Además cada nodo tiene a su derecha a un nodo con identificador menor, salvo el nodo con el menor id que tiene a su derecha el nodo con el mayor id.

Complejidad de comunicaciones



En el ejemplo:

El mensaje del nodo 1 viaja 1 vez

El mensaje del nodo 2 viaja 2 veces

El mensaje del nodo 3 viaja 3 veces

...

El mensaje del nodo 8 viaja 8 veces

En general:

El mensaje del nodo con el id más pequeño viaja 1 vez

El mensaje del nodo que le sigue al más pequeño viaja 2 veces

...

El mensaje del nodo con el id más alto viaja n veces

En total se envían: $1+2+\dots+n = n(n+1)/2$ mensajes de candidatura y

n mensajes de $\text{Electo}(k) \Rightarrow$ en total tenemos $O(n^2)$ mensajes

Complejidad temporal

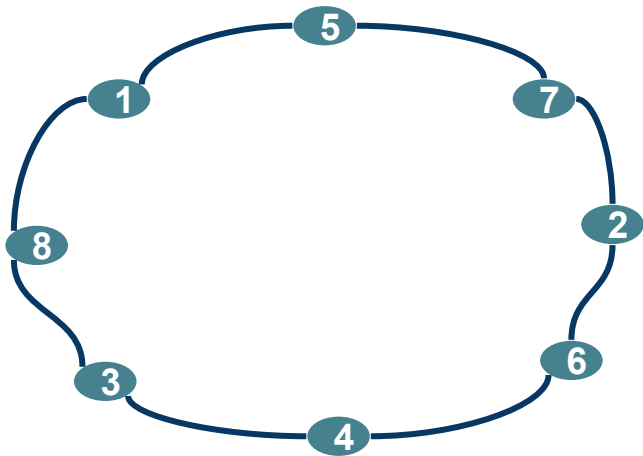
Lema 3. Asumiendo que cada mensaje se entrega con un retardo máximo de una unidad de tiempo, el algoritmo termina en $O(n)$ unidades de tiempo.

Peor caso: El único nodo que se despierta es el nodo que está a la derecha del nodo k .

- El mensaje viaja $n-1$ veces despertando a los nodos a su derecha hasta alcanzar al nodo k . Se requieren $n-1$ unidades de tiempo para esta etapa.
- El mensaje $\text{candidatura}(k)$ viaja n veces hasta regresar a k . Se requieren n unidades de tiempo para regresar esta etapa.
- El mensaje $\text{Electo}(k)$ se genera y se envía a viajar por la red. Se requieren n unidades de tiempo para esta etapa.

En total se necesitan: $3n-1$ unidades de tiempo. El algoritmo tiene una complejidad temporal $O(n)$ en el peor caso.

Algoritmo Elección Hitschberg- Sinclair



Modelo: Se considera un anillo pero esta vez los mensajes pueden ser transmitidos tanto a la derecha como a la izquierda sobre cualquier enlace. En otros términos el grafo es no dirigido.

Estrategia

- La contienda se lleva a cabo en rondas que abarcan una parte de la red (h vecinos a su izquierda y h vecinos a su derecha).
- El nodo ganador de la contienda en la ronda, pasa a competir a la siguiente ronda que abarca el doble de nodos de cada lado.
- La última ronda se juega en todo el anillo y el ganador final es el nuevo líder.



Variables de estado

- **estado**: despierto o dormido. Se usa para determinar si el nodo ya está al tanto de que hay una elección en curso. El valor inicial es dormido y cuando recibe un mensaje relativo a la elección cambia a despierto.
- **líder**: identificador del nodo que resultó electo. Al inicio tiene un valor no utilizado como id. En nuestro caso, 0.
- **vencidos**: registra el conocimiento que un nodo tiene sobre la proporción de nodos que ha vencido en la ronda actual. Al inicio es 0.



Mensajes

- **CANDIDATO($j, d, ronda$)** transporta:
 - j que es el identificador del nodo candidato
 - d que es la distancia en saltos desde el nodo en el que se originó
 - $ronda$ que es la ronda en la que se está compitiendo
- **REGRESA($j, ronda$)** transporta
 - j que es el identificador del ganador de la ronda
 - $ronda$ que es el número de ronda
- **ELECTO(j)** notifica la identidad del nodo que resultó electo.



Algoritmo Elección HS en nodo i

Al recibir CANDIDATO($j, \text{dist}, \text{ronda}$) desde dir

SI ($j > i$) ENTONCES

SI ($\text{dist} < 2^{\text{ronda}}$) ENTONCES

Envía CANDIDATO($j, \text{dist}+1, \text{ronda}$) a dir opuesta

OTRO SI ($\text{dist} = 2^{\text{ronda}}$) ENTONCES

Envía REGRESA(j, ronda) a dir

FINSI

OTRO SI ($j = i$) ENTONCES

Envía ELECTO(j) a la derecha

OTRO SI ($\text{estado} = \text{dormido}$) ENTONCES // $j < i$

inicia($i, 1, 0$)

FINSI

FINSI

estado = despierto

Algoritmo Elección HS en nodo i

Al recibir ELECTO(j) por la izquierda

 líder = j

 SI (j != i) ENTONCES

 Envía ELECTO(j) a la derecha

 estado = dormido

Algoritmo Elección HS en nodo i

Al recibir REGRESA(j,ronda) desde dir

SI (j != i) ENTONCES

Envía regresa(j,ronda) a dir opuesta

OTRO

vencidos += 0.5

SI (vencidos = 1) ENTONCES

inicia(i,1,ronda+1)

FINSI

FINSI

Algoritmo Elección HS en nodo i

PROCEDIMIENTO `inicia(id,distancia,ronda)`

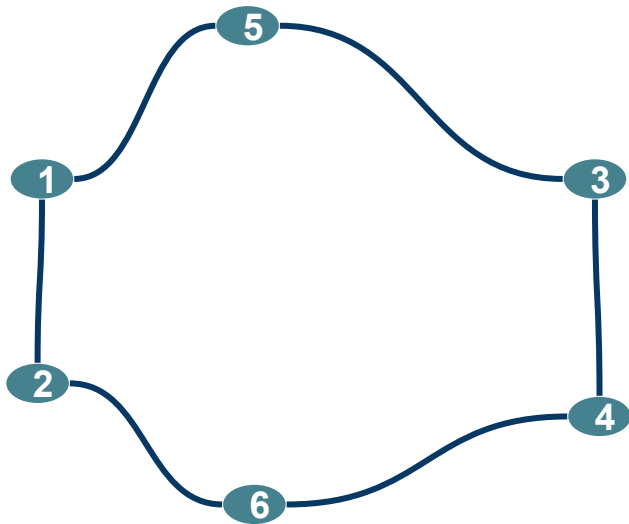
COMIENZA

Envía `CANDIDATO(id,distancia,ronda)` en ambas
Direcciones

`vencidos = 0`

TERMINA

Ejemplo de ejecución



Propiedades

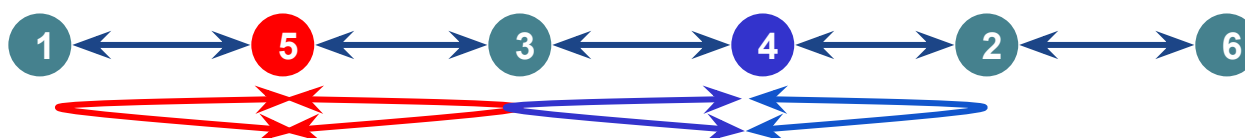
Lema 1. Todos los procesos que ejecutan el algoritmo elección HS terminan el algoritmo. Cuando esto sucede, existe exactamente un proceso al que todos reconocen como líder.



Propiedades

Lema 2. Suponiendo que en la ronda 0 inician n candidatos, para toda ronda $r \geq 1$ el número de procesos que llegan a la ronda r es a lo más $\lceil n/(2^{r-1}+1) \rceil$

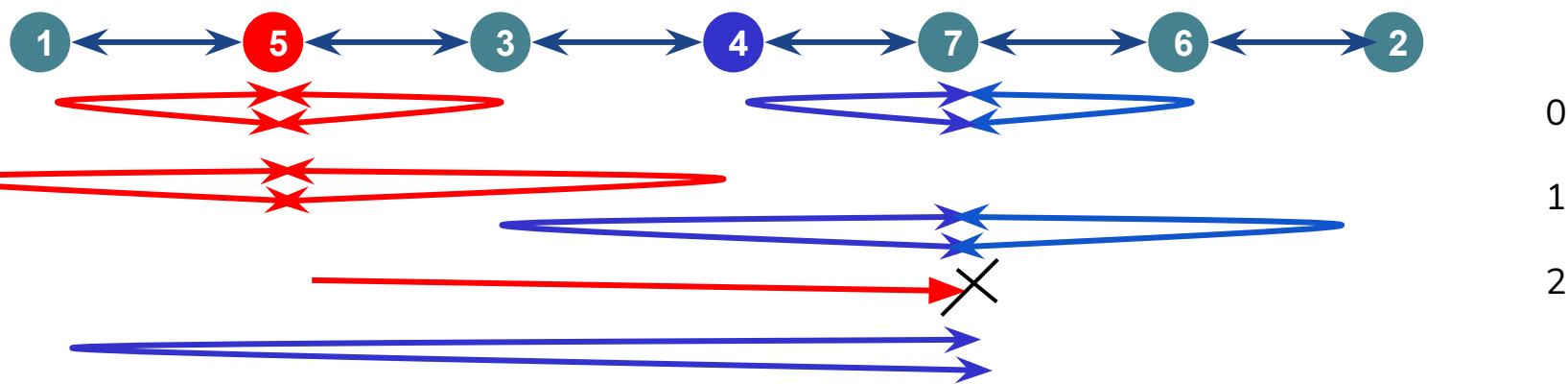
Demostración:



- Si 5 y 4 lanzan su candidatura, ambos ganarán la ronda 0 pues aún no se pueden “ver”. Sin embargo sólo uno sobrevive la siguiente ronda.

Propiedades

- En la siguiente configuración:

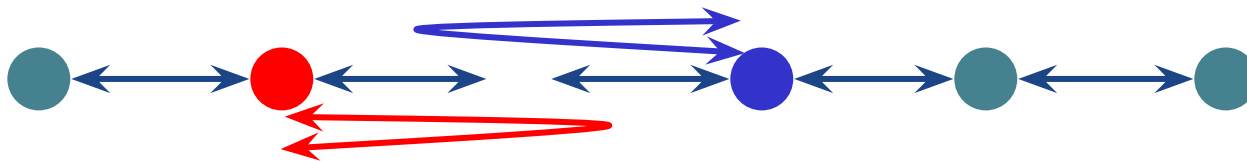


- Si 5 y 7 lanzan su candidatura, ambos ganarán la ronda 0 e incluso la ronda 1 pues aún no se pueden ver. Sin embargo sólo uno sobrevive la ronda 2.

Propiedades

Entonces para que dos candidatos sobrevivan:

- a) La ronda 0 $\Rightarrow$ tienen que estar a distancia al menos $2 = 2^0 + 1$
- b) La ronda 1 $\Rightarrow$ tienen que estar a distancia al menos $3 = 2^1 + 1$



En general: dos candidatos sobreviven la ronda x ssi están a distancia al menos $2^x + 1$.

$\Rightarrow$ al final de la ronda $r-1$ sobreviven los candidatos que están a distancia $2^{r-1} + 1$

$\Rightarrow$ a lo más puede haber uno por cada segmento del anillo de ese tamaño

$\Rightarrow$ Hay $\lceil n / (2^{r-1} + 1) \rceil$ segmentos y sobrevive un candidato por cada uno

$\Rightarrow$ A la ronda r llegan a lo mas $\lceil n / (2^{r-1} + 1) \rceil$ candidatos.

Complejidad en comunicaciones

- ¿Cuántas rondas se requieren?

Un nodo es electo cuando su candidatura le llega, es decir que da la vuelta completa, esto sucede en la ronda en donde $2^{\text{ronda}} \geq n$, y esta es la ronda final. Es decir se hacen R rondas donde

$$R = \lceil \log_2 n \rceil$$



Complejidad en comunicaciones

- ¿Cuántos mensajes se envían por ronda, asumiendo los n candidatos?

Para la ronda r un candidato envía su candidatura:

- a. a la izquierda a distancia 2^r
- b. a la derecha a distancia 2^r

Y la correspondiente transmisión del mensaje regresa

- c. por la izquierda desde una distancia 2^r
- d. por la derecha desde una distancia 2^r

⇒ En la ronda r se envían $4(2^r)$ mensajes por candidato.

⇒ como hay $\lceil n/(2^{r-1}+1) \rceil$ candidatos

⇒ se envían $4(2^r)\lceil n/(2^{r-1}+1) \rceil = 8(2^r)\lceil n/(2^r+2) \rceil < 8n = O(n)$

Complejidad en comunicaciones

Corolario 1. El total de mensajes transmitidos es $O(n \log n)$

El algoritmo efectúa R rondas donde:

$$2^{R-1} < n \leq 2^R \Rightarrow R = \lceil \log_2 n \rceil$$

En cada ronda se envían una cantidad de mensajes acotada por $8n$, entonces, en total:

$$\underbrace{\lceil \log_2 n \rceil * 8n}_{\text{candidatura y regresa}} + \underbrace{n}_{\text{electo}} \in O(n \log n)$$

Complejidad temporal

Corolario 2. Asumiendo que cada mensaje se entrega en una unidad de tiempo, el algoritmo concluye en un plazo $O(n)$.

La ronda 0 toma 2 unidades de tiempo

La ronda 1 toma 4 unidades de tiempo

...

La ronda r toma $2 \cdot 2^r$ unidades de tiempo

$\Rightarrow$ en total las R rondas de contienda toman:

$$2(2^0 + 2^1 + \dots + 2^{R-1} + 2^R) = 2(2^{R+1} - 1) = 2(2^{\lceil \log_2 n \rceil + 1} - 1) = 2(2 * 2^{\lceil \log_2 n \rceil} - 1) < 2(4n - 1) = O(n)$$

$\Rightarrow$ + n unidades de tiempo para transmitir electo(j)

$\Rightarrow O(n)$ unidades de tiempo

